

procedure

dd896eca-ea7d-4323-aaab-72118867fca9.pdf

PDF

create tables and answer the questions in one sql server

```
SQL
-- =====
-- DATABASE CREATE
-- =====
CREATE DATABASE BankDB;
GO

USE BankDB;
GO

-- =====
-- TABLE CREATE
-- =====

CREATE TABLE Zone
(
    Zone_ID INT PRIMARY KEY,
    Zone_Name VARCHAR(50)
);

CREATE TABLE Branch
(
    Branch_ID INT PRIMARY KEY,
    Branch_Name VARCHAR(50),
    Zone_ID INT,
    FOREIGN KEY (Zone_ID) REFERENCES Zone(Zone_ID)
);

CREATE TABLE AccountHolder
(
    Acc_ID INT PRIMARY KEY,
    Acc_Holder_Name VARCHAR(100),
    Amount MONEY,
    Branch_ID INT,
    FOREIGN KEY (Branch_ID) REFERENCES Branch(Branch_ID)
);

-- =====
-- INSERT DATA
-- =====

INSERT INTO Zone VALUES
(1, 'Rajshahi'),
(2, 'Dhaka'),
(3, 'Khulna');

INSERT INTO Branch VALUES
(101, 'Bangla Branch', 1),
(102, 'Basic Branch', 1),
(103, 'Banani Branch', 2),
(104, 'City Branch', 2),
(105, 'Bagerhat Branch', 3);

INSERT INTO AccountHolder VALUES
(1, 'Mr. Rahim', 12000, 101),
(2, 'Mr. Karim', 25000, 102),
(3, 'Mrs. Jui', 18000, 103),
(4, 'Mr. Hasan', 35000, 104),
(5, 'Mr. Tanvir', 9000, 105),
(6, 'Ms. Mim', 15000, 101),
(7, 'Mr. Robin', 28000, 103);

-- =====
-- 1. SPdetails
-- Find Acc_holder_name, Amount, Branch_Name and Zone_Name
-- =====

CREATE PROCEDURE SPdetails
AS
BEGIN
    SELECT
        A.Acc_Holder_Name,
        A.Amount,
        B.Branch_Name,
        Z.Zone_Name
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID;
END;
GO

EXEC SPdetails;
GO

-- =====
-- 2. SPaverage
-- Find Branch_Name and Amount greater than parameter amount
-- =====

CREATE PROCEDURE SPaverage
    @Amount MONEY
AS
BEGIN
    SELECT
        B.Branch_Name,
        A.Amount
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    WHERE A.Amount > @Amount;
END;
GO

EXEC SPaverage 17000;
GO

-- =====
-- 3. SPbalance
-- Find total amount of particular zone using RETURN
-- =====

CREATE PROCEDURE SPbalance
    @ZoneName VARCHAR(50)
AS
BEGIN
    DECLARE @Total INT;

    SELECT @Total = SUM(A.Amount)
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID
    WHERE Z.Zone_Name = @ZoneName;

    RETURN @Total;
END;
GO

DECLARE @Result INT;

EXEC @Result = SPbalance 'Rajshahi';
```

```

PRINT 'Total Amount = ' + CAST(@Result AS VARCHAR);
GO

-- =====
-- 4. SPamount
-- Find holders whose name has 'Mr.'
-- and amount less than maximum amount
-- =====

CREATE PROCEDURE SPamount
AS
BEGIN
    SELECT
        A.Acc_Holder_Name,
        B.Branch_Name,
        Z.Zone_Name,
        A.Amount
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID
    WHERE A.Acc_Holder_Name LIKE 'Mr.%'
    AND A.Amount < (SELECT MAX(Amount) FROM AccountHolder);
END;
GO

EXEC SPamount;
GO

-- =====
-- 5. SPdetailsInfo
-- Find number of customer of each Zone
-- Output parameter
-- =====

CREATE PROCEDURE SPdetailsInfo
@ZoneName VARCHAR(50),
@CustomerCount INT OUTPUT
AS
BEGIN
    SELECT @CustomerCount = COUNT(A.Acc_ID)
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID
    WHERE Z.Zone_Name = @ZoneName;
END;
GO

DECLARE @Count INT;

EXEC SPdetailsInfo 'Dhaka', @Count OUTPUT;

PRINT 'Number of Customer = ' + CAST(@Count AS VARCHAR);
GO

-- =====
-- 6. spEmployeeSalaryDetails1
-- Find number of Branch_Name whose amount
-- between range and Branch_Name contains 'Ba'
-- =====

CREATE PROCEDURE spEmployeeSalaryDetails1
@StartAmount MONEY,
@EndAmount MONEY,
@BranchName VARCHAR(50),
@BranchCount INT OUTPUT
AS
BEGIN
    SELECT @BranchCount = COUNT(DISTINCT B.Branch_Name)
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    WHERE A.Amount BETWEEN @StartAmount AND @EndAmount
    AND B.Branch_Name LIKE '%' + @BranchName + '%';
END;
GO

DECLARE @TotalBranch INT;

EXEC spEmployeeSalaryDetails1
7000,
30000,
'Ba',
@TotalBranch OUTPUT;

PRINT 'Total Branch = ' + CAST(@TotalBranch AS VARCHAR);
GO

-- =====
-- 7. SPdetailsInfoZone
-- Find Zone_Name and number of customer
-- of a specific zone
-- =====

CREATE PROCEDURE SPdetailsInfoZone
@ZoneName VARCHAR(50)
AS
BEGIN
    SELECT
        Z.Zone_Name,
        COUNT(A.Acc_ID) AS Number_Of_Customer
    FROM AccountHolder A
    JOIN Branch B
        ON A.Branch_ID = B.Branch_ID
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID
    WHERE Z.Zone_Name = @ZoneName
    GROUP BY Z.Zone_Name;
END;
GO

EXEC SPdetailsInfoZone 'Rajshahi';
GO

-- =====
-- 8. SPdetailsInfo1
-- Find Zone_Name and number of Branch
-- Branch name pass by parameter
-- =====

CREATE PROCEDURE SPdetailsInfo1
@BranchName VARCHAR(50)
AS
BEGIN
    SELECT
        Z.Zone_Name,
        COUNT(B.Branch_ID) AS Number_Of_Branch
    FROM Branch B
    JOIN Zone Z
        ON B.Zone_ID = Z.Zone_ID
    WHERE B.Branch_Name LIKE '%' + @BranchName + '%'
    GROUP BY Z.Zone_Name;
END;
GO

EXEC SPdetailsInfo1 'Ba';
GO

```

give me full details about procedure

Stored Procedure in SQL Server

A **Stored Procedure** is a precompiled collection of SQL statements stored inside the database. It is used to perform a specific task repeatedly.

In SQL Server, procedures help to:

- Reduce code repetition
- Increase speed
- Improve security
- Make programs easier to maintain

Basic Syntax of Procedure

```
SQL
CREATE PROCEDURE ProcedureName
AS
BEGIN
    SQL Statements
END;
```

Example 1: Simple Procedure

Create Table

```
SQL
CREATE TABLE Student
(
    ID INT,
    Name VARCHAR(50),
    Department VARCHAR(50)
);
```

Insert Data

```
SQL
INSERT INTO Student VALUES
(1,'Rahim','CSE'),
(2,'Karim','EEE'),
(3,'Jui','BBA');
```

Create Procedure

```
SQL
CREATE PROCEDURE SPStudent
AS
BEGIN
    SELECT * FROM Student;
END;
```

Execute Procedure

```
SQL
EXEC SPStudent;
```

OR

```
SQL
EXECUTE SPStudent;
```

Explanation

Part	Meaning
CREATE PROCEDURE	Used to create procedure
SPStudent	Procedure name
AS	Starts procedure body
BEGIN END	Procedure block
SELECT * FROM Student	SQL query

Procedure With Parameter

Parameters allow dynamic values.

Example 2: Input Parameter

```
SQL
CREATE PROCEDURE SPStudentByDept
@Dept VARCHAR(50)
AS
BEGIN
    SELECT *
    FROM Student
    WHERE Department = @Dept;
END;
```

Execute

```
SQL
EXEC SPStudentByDept 'CSE';
```

Output

Shows only CSE students.

Multiple Parameters

```
SQL
CREATE PROCEDURE SPStudentInfo
@ID INT,
@Dept VARCHAR(50)
AS
BEGIN
    SELECT *
```

```
FROM Student
WHERE ID = @ID
AND Department = @Dept;
END;
```

Execute

```
SQL
EXEC SPStudentInfo 1, 'CSE';
```

Output Parameter

Used to return value from procedure.

Example 3

```
SQL
CREATE PROCEDURE SPCountStudent
@Dept VARCHAR(50),
@Total INT OUTPUT
AS
BEGIN
SELECT @Total = COUNT(*)
FROM Student
WHERE Department = @Dept;
END;
```

Execute

```
SQL
DECLARE @Result INT;
EXEC SPCountStudent 'CSE', @Result OUTPUT;
PRINT @Result;
```

Explanation

Keyword	Meaning
OUTPUT	Returns value
DECLARE	Creates variable
PRINT	Shows result

RETURN Statement

RETURN sends a single integer value.

Example 4

```
SQL
CREATE PROCEDURE SPReturnExample
AS
BEGIN
RETURN 100;
END;
```

Execute

```
SQL
DECLARE @Value INT;
EXEC @Value = SPReturnExample;
PRINT @Value;
```

ALTER Procedure

Used to modify existing procedure.

Syntax

```
SQL
ALTER PROCEDURE ProcedureName
AS
BEGIN
SQL Statements
END;
```

Example

```
SQL
ALTER PROCEDURE SPStudent
AS
BEGIN
SELECT Name, Department
FROM Student;
END;
```

DROP Procedure

Deletes procedure permanently.

Syntax

```
SQL
DROP PROCEDURE ProcedureName;
```

Example

```
SQL
DROP PROCEDURE SPStudent;
```

Procedure With IF ELSE

```
SQL
CREATE PROCEDURE SPMarks
@Marks INT
AS
BEGIN
    IF @Marks >= 40
        PRINT 'PASS';
    ELSE
        PRINT 'FAIL';
END;
```

Execute

```
SQL
EXEC SPMarks 75;
```

Procedure With WHILE Loop

```
SQL
CREATE PROCEDURE SPLoop
AS
BEGIN
    DECLARE @X INT = 1;

    WHILE @X <= 5
    BEGIN
        PRINT @X;
        SET @X = @X + 1;
    END
END;
```

Procedure With INSERT

```
SQL
CREATE PROCEDURE SPInsertStudent
@ID INT,
@Name VARCHAR(50),
@Dept VARCHAR(50)
AS
BEGIN
    INSERT INTO Student
VALUES(@ID,@Name,@Dept);
END;
```

Execute

```
SQL
EXEC SPInsertStudent 4, 'Hasan', 'CSE';
```

Procedure With UPDATE

```
SQL
CREATE PROCEDURE SPUpdateStudent
@ID INT,
@Dept VARCHAR(50)
AS
BEGIN
    UPDATE Student
SET Department = @Dept
WHERE ID = @ID;
END;
```

Execute

```
SQL
EXEC SPUpdateStudent 2, 'CSE';
```

Procedure With DELETE

```
SQL
CREATE PROCEDURE SPDeleteStudent
@ID INT
AS
BEGIN
    DELETE FROM Student
WHERE ID = @ID;
END;
```

Execute

```
SQL
EXEC SPDeleteStudent 3;
```

Advantages of Procedure

Advantage	Description
Faster Execution	Precompiled
Reusable	Write once use many times
Secure	Can restrict table access
Easy Maintenance	Easy to update
Reduces Network Traffic	One call executes many queries

Disadvantages

Disadvantage	Description
Hard Debugging	Complex procedures difficult
More Memory	Stored in database
Dependency	Changes may affect apps

System Stored Procedures

SQL Server has built-in procedures.

Examples:

```
SQL
sp_help

SQL
sp_tables

SQL
sp_columns Student
```

View All Procedures

```
SQL
SELECT * FROM sys.procedures;
```

Rename Procedure

```
SQL
EXEC sp_rename 'OldProcedureName', 'NewProcedureName';
```

Encrypted Procedure

```
SQL
CREATE PROCEDURE SPTest
WITH ENCRYPTION
AS
BEGIN
SELECT * FROM Student;
END;
```

Procedure vs Function

Procedure	Function
Can return many values	Returns one value
Can use INSERT/UPDATE/DELETE	Mostly used for calculations
Called by EXEC	Used inside SELECT
Can have OUTPUT	Must return value

Complete Simple Example

```
SQL
CREATE DATABASE CollegeDB;
GO

USE CollegeDB;
GO

CREATE TABLE Student
(
    ID INT,
    Name VARCHAR(50),
    Department VARCHAR(50)
);

INSERT INTO Student VALUES
(1,'Rahim','CSE'),
(2,'Karim','EEE'),
(3,'Jui','BBA');

CREATE PROCEDURE SPStudent
AS
BEGIN
SELECT * FROM Student;
END;

EXEC SPStudent;
```